ExplAIner

3º ano · Técnico em Informática · COTEMIG

O ExplAIner atende **estudantes do Ensino Médio** e ajuda a resolver a **falta de prática guiada e de rotina de estudos**.

INTEGRANTES

Álvaro Pires de Souza

Antônio Henrique de Paula Reis

Henrique Saddi Meinicke

Josué Fernando Silva Melo

Lucas Ribeiro Fontana

Weber Anjos de Souza Sodré

PROJETO E PÚBLICO-ALVO

Estudar sozinho é difícil de manter

Tempo perdido

procurando um conteúdo bom para estudar rápido

Respostas genéricas

de IA, sem correção nem explicação

Falta de rotina

o aluno não mantém o hábito de estudar

Solução: uma jornada de estudos em fases, com questões criadas por IA, correção na hora, pontos, ranking e sequência de dias.

Quem usa: estudantes do Ensino Médio.

CASO DE USO 1 DE 5

Criar conta e aceitar os termos



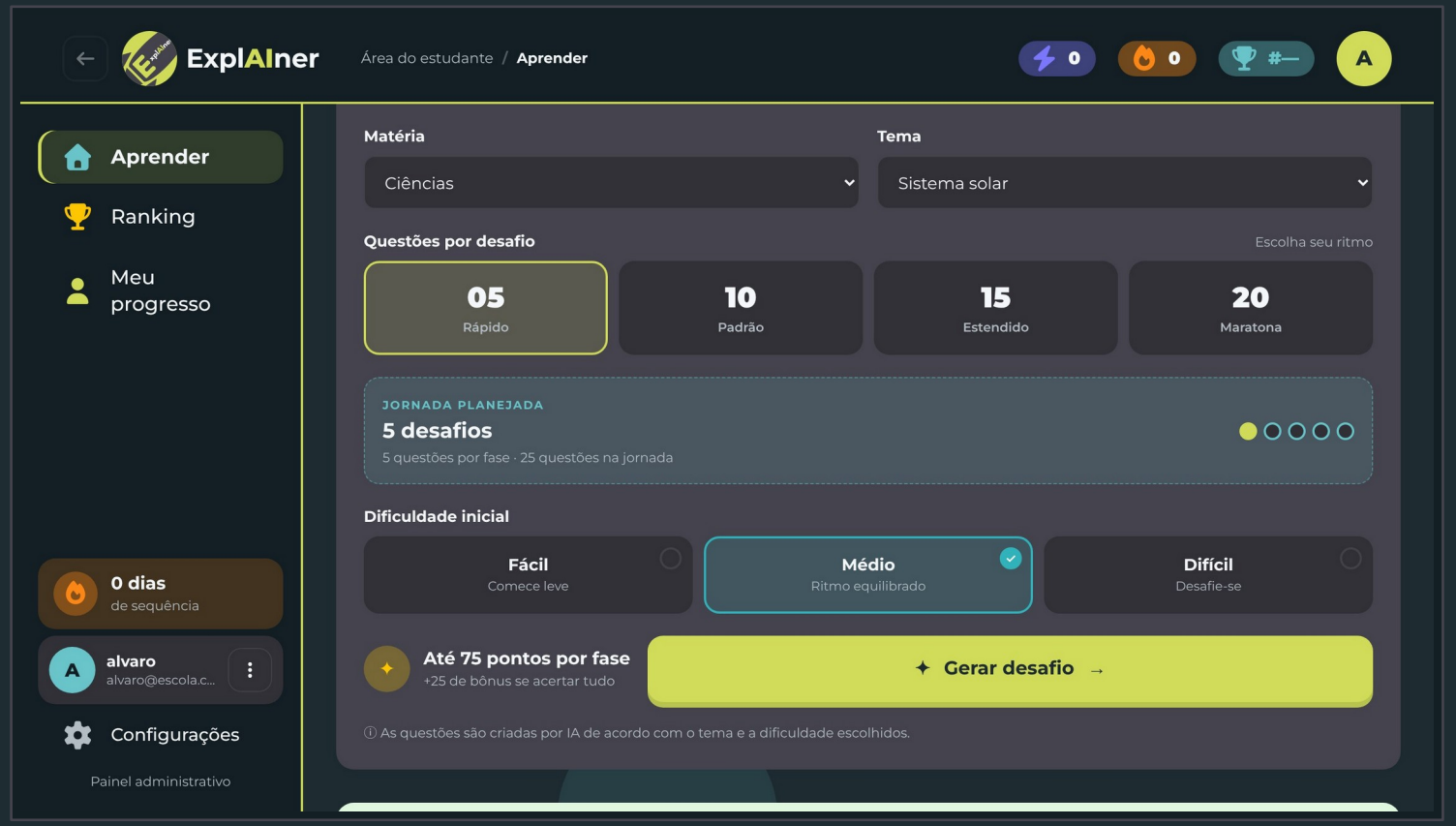
OBJETIVO

O aluno cria a conta com nome, nick, e-mail e senha para começar a estudar.

- Senha guardada com hash
- Sem aceite dos termos, a conta não é criada

CASO DE USO 2 DE 5

Gerar desafio por tema e dificuldade



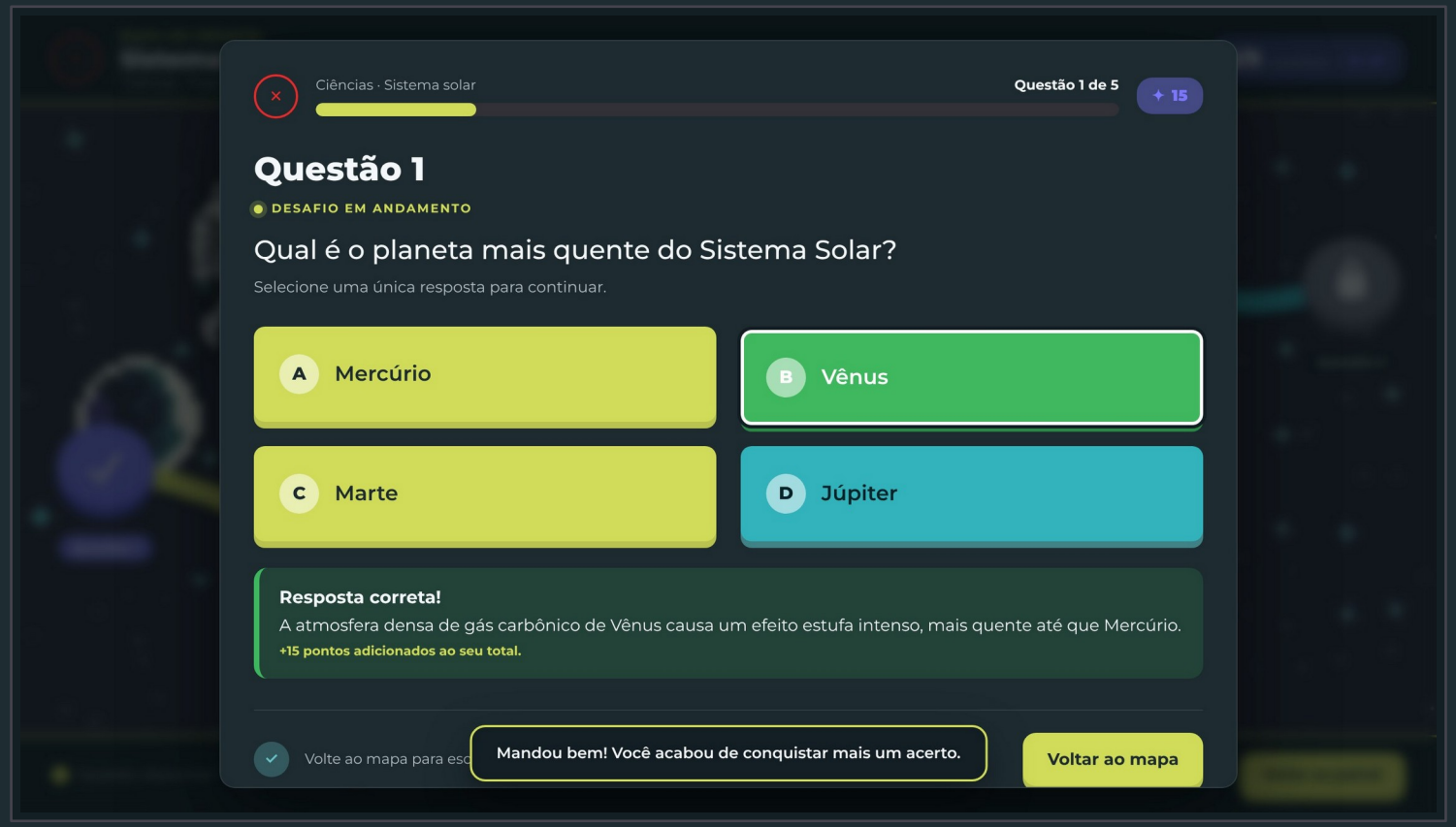
OBJETIVO

O aluno escolhe matéria, tema, quantidade e dificuldade; a IA cria as questões da fase.

- 5, 10, 15 ou 20 questões por fase
- Fácil, Médio ou Difícil

CASO DE USO 3 DE 5

Responder e receber a correção



OBJETIVO

O aluno responde em ordem e vê na hora se acertou, a explicação e os pontos.

- Verde no acerto, vermelho no erro
- Pontos conforme a dificuldade

CASO DE USO 4 DE 5

Concluir a fase e avançar



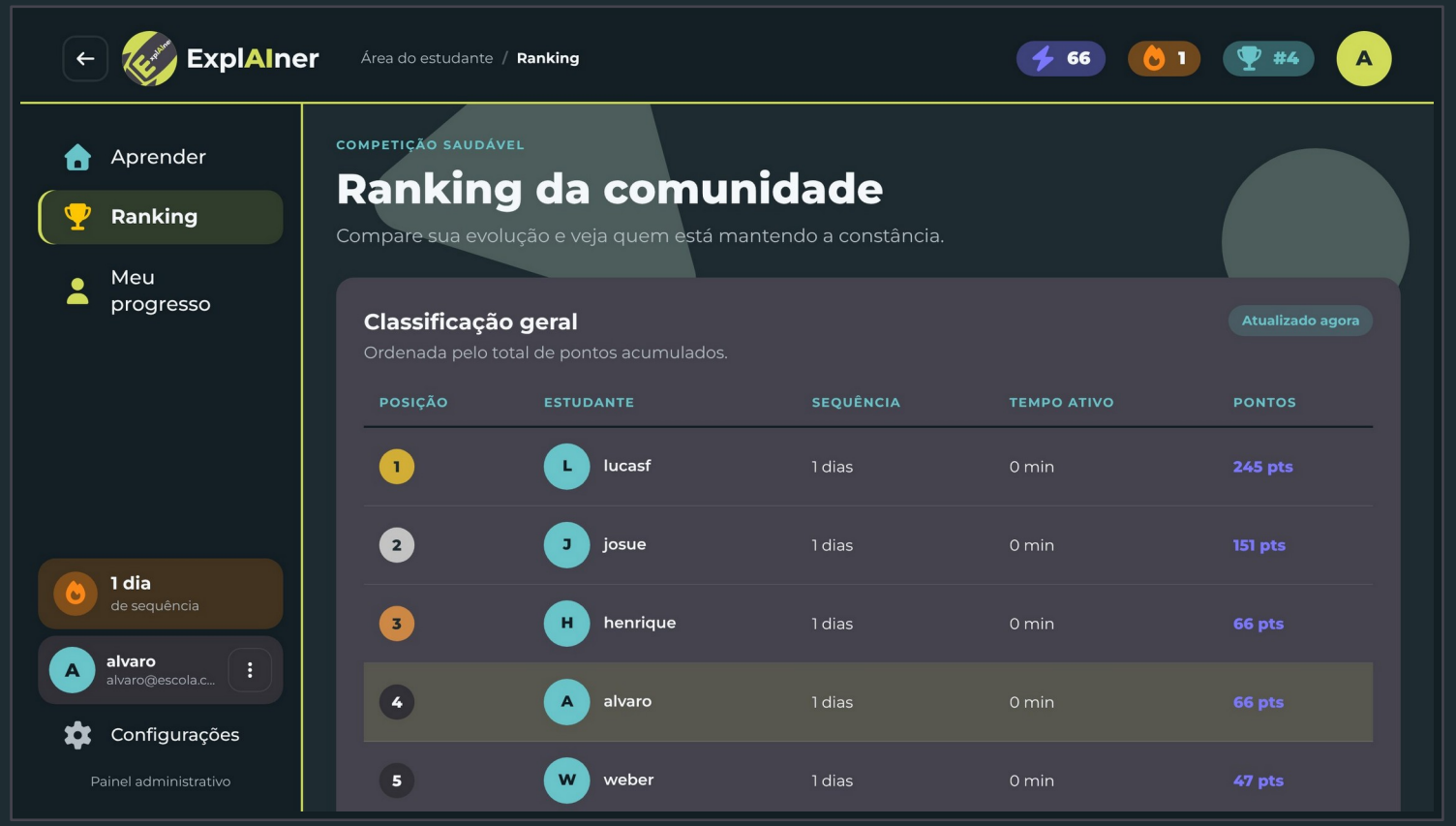
OBJETIVO

O aluno fecha a fase, recebe os bônus e libera a próxima, mais difícil.

- +10 fase, +25 perfeita, +50 jornada
- 70% de erros: a jornada recomeça

CASO DE USO 5 DE 5

Acompanhar ranking e progresso



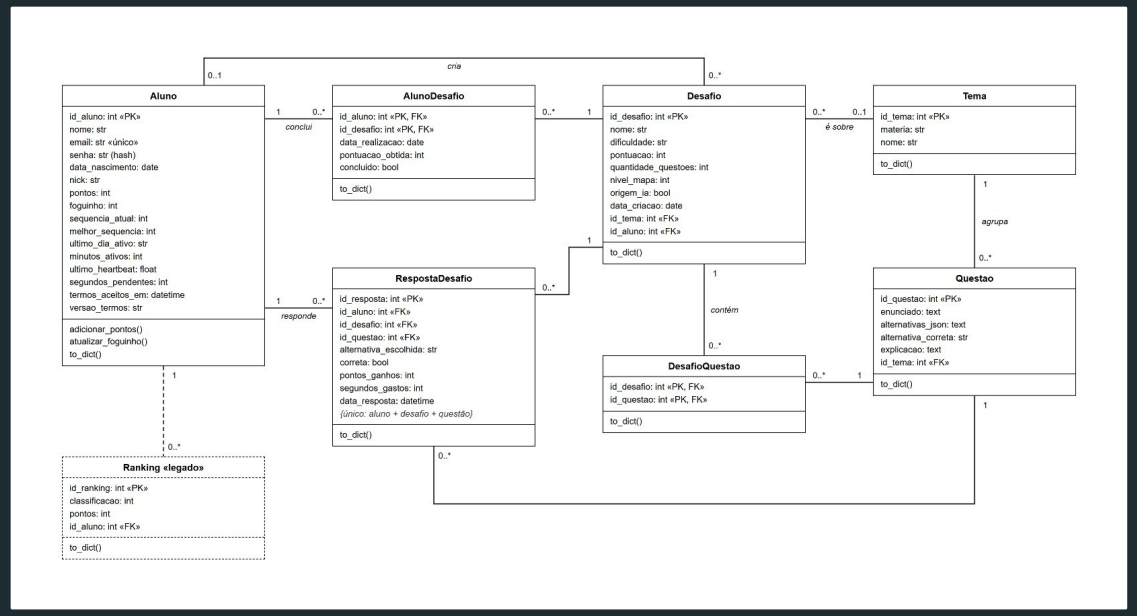
OBJETIVO

O aluno compara seus pontos com os colegas e acompanha a evolução.

- Posição, pontos e foguinho
- Histórico com opção de refazer

MODELO DE DOMÍNIO

O aluno joga desafios e cada resposta é salva



Classes principais

Aluno, Desafio, Questao e RespostaDesafio; Tema organiza questões e desafios

Relacionamentos

Desafio e Questao são N:N pela DesafioQuestao; RespostaDesafio liga Aluno, Desafio e Questao

Conclusão de fase

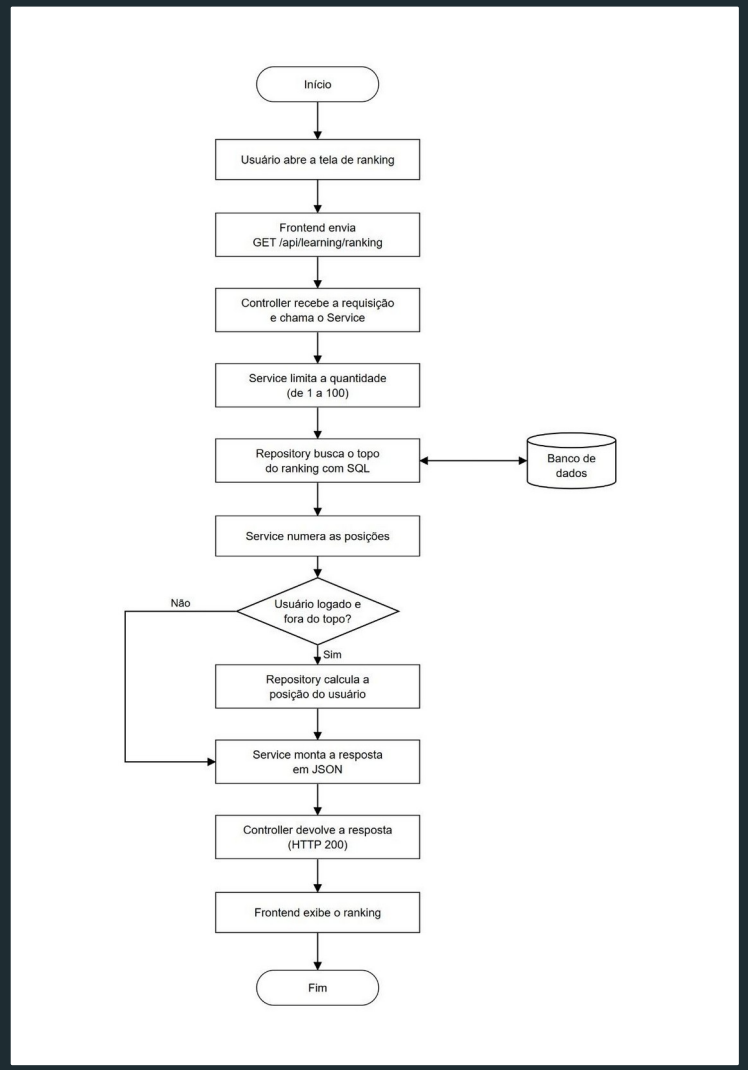
AlunoDesafio guarda quando a fase foi concluída e os pontos

No código

cada classe é um Model em backend/models e vira uma tabela no banco

FLUXOGRAMA · RECUPERAÇÃO DE DADOS

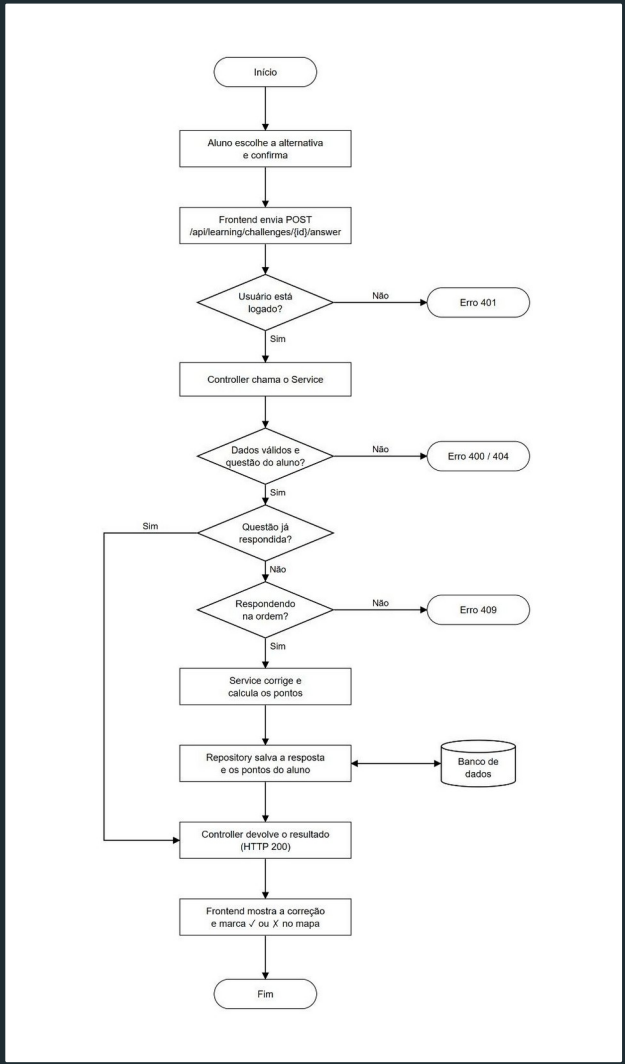
Ranking: o Repository busca no banco com SQL



Frontend	GET /api/learning/ranking
Controller	ranking_de_estudo() recebe e chama o Service
Service	ProgressoService limita de 1 a 100 e numera as posições
Repository	AlunoRepository.ranking_topo() faz o SELECT
Resposta	JSON com posições, pontos e total

FLUXOGRAMA · ENTRADA DE DADOS

Responder: o Repository grava a resposta e os pontos



Frontend	POST /api/learning/challenges/{id}/answer
Controller	confere o login e chama o Service
Service	RespostaService valida e calcula os pontos
Repository	RespostaRepository.adicionar() e commit
Banco	INSERT da resposta e UPDATE dos pontos

ARQUITETURA E ORGANIZAÇÃO DO CÓDIGO

Quatro camadas no backend

1

Controllers

backend/controllers/

recebem a requisição e devolvem JSON

2

Services

backend/services/

executam o caso de uso e as regras

3

Repositories

backend/repositories/

fazem as operações e consultas no banco

4

Models

backend/models/

representam as tabelas

Frontend separado: a pasta frontend/ (estudar.html e estudar.js) não acessa o banco; ela só chama a API.

UM FLUXO DO SISTEMA · GERAR DESAFIO

A requisição passa por todas as camadas

1	**Controller**	criar_desafio_jogavel() recebe o pedido de novo desafio e confere se o aluno está logado
2	**Service**	DesafioService.criar() valida tema, quantidade, dificuldade e fase e chama a IA
3	**Repository**	DesafioRepository, QuestaoRepository e DesafioQuestaoRepository gravam o desafio, as questões e os vínculos
4	**Model**	Desafio, Questao e DesafioQuestao representam os dados
5	**Resposta**	o JSON volta ao estudar.js, que desenha o mapa da fase

SEPARAÇÃO ENTRE FRONTEND E API

O frontend consome a API por requisições

frontend/estudar.js · a tela pede os dados

```
async function api(caminho, opcoes) {
  const resposta = await fetch(
    API_URL + caminho, configuracao);
  return resposta.json();
}

api("/api/learning/ranking");
```



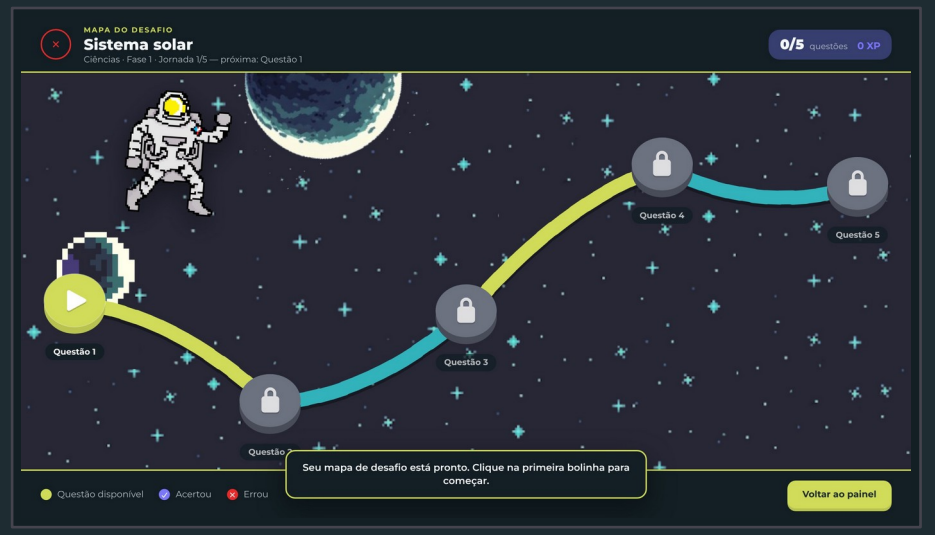
GET /api/learning/ranking · a API responde


```
{
  "itens": [
    { "posicao": 1,
      "nick": "lucasf",
      "pontos": 245 },
    { "posicao": 2,
      "nick": "josue",
      "pontos": 151 }
  ],
  "total_alunos": 6
}
```

DEMONSTRAÇÃO · ATÉ 2 MINUTOS

Ação do aluno → resultado do sistema

1	**Gera um desafio com IA**	→ as questões são criadas e o mapa abre
2	**Responde as questões**	→ verde ou vermelho, explicação e pontos
3	**Conclui a fase**	→ bônus e próxima fase mais difícil
4	**Sai e volta**	→ continua de onde parou
5	**Abre o ranking**	→ posição, pontos e foguinho



Mapa do desafio

PERSISTÊNCIA E CONSULTAS

ORM no CRUD e SQL só no Repository

CRUD com ORM (SQLAlchemy)

Cadastro

```
db.session.add(aluno)
```

Login

```
Aluno.query.filter_by(
    email=email).first()
```

Perfil

```
aluno.nick = nick
```

Excluir conta

```
db.session.delete(aluno)
db.session.commit()
```

SQL em repositories/aluno_repositories.py

```
SELECT a.id_aluno, a.nick,
  COALESCE(a.pontos, 0) AS pontos
FROM {aluno} a
WHERE a.id_aluno IN (
  SELECT DISTINCT r.id_aluno
  FROM resposta_desafio r)
ORDER BY pontos DESC, a.nome
LIMIT :limite
```

README

Os casos de uso implementados estão no README

README.md · Casos de uso implementados · github.com/lucasf0ntana/ExplAIner-Final

Nº	Caso de uso	Ator	Tela
UC01	**Criar conta e aceitar os termos**	Aluno	Login / Criar conta
UC02	**Entrar e sair da conta**	Aluno	Login
UC03	**Gerar desafio por tema e dificuldade**	Aluno	Aprender
UC04	**Responder questão**	Aluno	Mapa do desafio / Questão
UC05	**Concluir fase e avançar na jornada**	Aluno	Resultado da fase
UC06	**Continuar desafio em andamento**	Aluno	Aprender
UC07	**Refazer fase concluída**	Aluno	Meu progresso
UC08	**Consultar ranking**	Aluno	Ranking
UC09	**Acompanhar progresso e histórico**	Aluno	Meu progresso
UC10	**Manter a sequência de estudos (foguinho)**	Aluno	Todas (contagem automática)
UC11	**Alterar nick**	Aluno	Meu progresso
UC12	**Aceitar termos em conta existente**	Aluno	Modal de termos
UC13	**Excluir a própria conta**	Aluno	Menu da conta
UC14	**Gerenciar cadastros (CRUD)**	Administrador	Painel administrativo

TECNOLOGIAS E BÔNUS

IA, hospedagem e termos registrados

TECNOLOGIAS

- Python e Flask
- SQLAlchemy (ORM)
- SQLite e PostgreSQL
- HTML, CSS e JavaScript
- API do Google Gemini
- Render e pytest

BÔNUS



IA como serviço
o Google Gemini cria as questões do tema escolhido



Frontend e API hospedados
no Render, com banco PostgreSQL



Termos com decisão registrada
data e versão do aceite salvas no banco



Obrigado!

Prática guiada, correção na hora e motivo para voltar todo dia.

Perguntas?